

Aula 5 — Maestro E2E em RN + Lab Perf/Security + Trabalho Final

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 02/07/2026

Agenda de hoje (3h30)

1. **Plantão de setup + Lab perf/security** — rodam em paralelo (~40min)
2. Recap da pirâmide de testes
3. **Maestro sobre o app RN** — fechar os 5 flows (Atividade 4)
4. Intervalo
5. **Trabalho Final** — revelar temas + rubrica
6. Combinar a data da apresentação

Como vamos usar os próximos ~40min — 2 caminhos

Com device/emulador de pé? Começa o **lab de perf/security**, sozinho:

```
exercicios/03-maestro-e2e/lab-perf-security.md
```

Sem device ainda? Prioridade é **configurar** — chama o prof, resolvemos juntos com calma.
Android Studio, emulador ou cabo/celular físico.

Todo mundo com algo pra fazer em paralelo. Reconvergemos em ~40min.

O app-alvo: CineFav

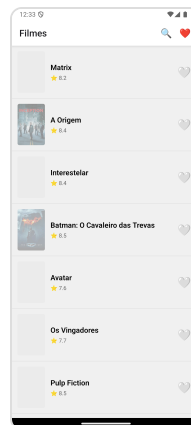
`com.puciec.cinefav` — abre na tela de **Login**.

Login mock: **qualquer email com @** + **senha 4+ caracteres**.

Ex.: `aluno@puc.br` / `1234`.

Dados **mockados** por padrão (sem token TMDB, sem rede) — flows **determinísticos**.

Matrix sempre existe.



Depois do login: é essa tela (lista de filmes) que a maioria dos flows valida.

Lab — Performance + Security no CineFav

Self-guided. Cada conceito amarrado ao app que vocês já têm rodando — sem painel genérico de ferramenta.

0. Antes de começar

`adb shell ...` roda **de qualquer pasta** — só precisa do device conectado. Só o que mexe em arquivo do repo (manifest, `./gradlew`) exige `cd exercicios/03-maestro-e2e/pratica`.

```
adb devices    # esperado: "emulator-5554 device" (ou serial do celular)
```

Travou? `adb devices` vazio → reabre emulador/religa cabo. `adb: command not found` → falta `ANDROID_HOME` no PATH (ver `COMECE-AQUI.md`). Comando trava → `adb kill-server && adb start-server`.

Performance — cold start, o que é e por que importa

App já **instalado** (feito no onboarding). Se não estiver: o `CineFav.apk` fica onde você **baixou** (não no repo) — `cd ~/Downloads` (ou onde salvou) antes de instalar:

```
adb install -r CineFav.apk
```

Cold start = o app não tinha processo rodando — Android cria tudo do zero (processo, memória, primeira tela). É o pior caso e o que o usuário sente na cara ao abrir o app pela 1ª vez ou depois de horas fechado. Ref.: [Android Developers — app startup](#).

Performance — medindo o cold start

```
adb shell am force-stop com.puciec.cinefav  
adb shell am start -W com.puciec.cinefav/.MainActivity
```

Olhar **TotalTime** (ms). Rodamos 3x agora:

```
TotalTime: 1935  
TotalTime: 2612  
TotalTime: 3521
```

Variação normal em emulador (recursos da máquina host competindo).

Warm start não é só "tirar o force-stop" — se o app ainda tá aberto, **TotalTime** vem 0 (não mede nada).
Manda pra background primeiro: `adb shell input keyevent KEYCODE_HOME`, depois roda o `am start` de novo
(processo já existe na memória = **warm start**, bem mais rápido).

Performance — jank ao rolar (quem tem device/emulador)

Jank = frame que estourou o orçamento de 16ms (60fps). Cada frame atrasado é um "solução" visual ao rolar — o usuário não sabe o termo técnico, mas sente o app como travado/de baixa qualidade, mesmo funcionando 100% certo.

Rolar a lista de filmes no device, depois:

```
adb shell dumpsys gfxinfo com.puciec.cinefav
```

Olhar **Janky frames** (% acima de 16ms/60fps). Rodamos agora:

```
Total frames rendered: 67  
Janky frames: 66 (98.51%)
```

Número alto é esperado sem GPU real (emulador) — não é o app. Comparem com quem tá em device físico (bem menor).

Performance — sentindo uma regressão de verdade (bônus)

O CineFav não tem gargalo real pra "achar" — mas o script `scripts/toggle-perf-regression.js` liga um delay **sintético e didático** (marcado `PERF-DEMO` no código, não é achado real como o `allowBackup`) pra vocês sentirem o antes/depois com número medido, não estimado:

```
node scripts/toggle-perf-regression.js           # liga delay de 1500ms no boot
cd android && ./gradlew assembleDebug && cd .. && adb install -r android/app/build/outputs/apk/debug/app-debug.apk
adb shell am force-stop com.puciec.cinefav && adb shell am start -W com.puciec.cinefav/.MainActivity
```

Resultado real (cold start, testado agora, mesmo emulador, 3 execuções cada):

- **Sem regressão:** 4808 – 6033ms
- **Com regressão (+1500ms artificial):** 10477 – 19643ms

O achado interessante não é o número, é o **formato**: 1500ms de `Thread.sleep` virou +5000 a +14000ms de `TotalTime` — não é soma linear. Bloquear a **main thread** no boot atrasa em cascata tudo que depende dela. Por isso "não bloquear a main thread" é regra de ouro, não recomendação teórica. **Reverter**: rodar o script de novo (desliga) → rebuild → reinstalar.

Security — o que `allowBackup` controla, de verdade

`android:allowBackup="true"` é o app dizendo "sistema, pode exportar meus dados sem eu perguntar."
Historicamente (Android <12), bastava isso pra `adb backup` copiar o sandbox inteiro do app — **sem senha, sem root**.

Testamos agora, de verdade, no nosso CineFav (build debug, emulador Android 15):

```
adb backup -f cinefav.ab com.puciec.cinefav  
# confirma "Back up my data" na tela do device
```

Resultado real: arquivo de 1.2MB, sem senha, sem root. Descompactando (`zlib` + `tar`), sai o sandbox inteiro do app.

Security — o que sai do backup, de verdade

```
$ tar -tvf cinefav.tar | grep -E "sp/|webview"
apps/com.puciec.cinefav/sp/WebViewChromiumPrefs.xml
apps/com.puciec.cinefav/r/app_webview/Default/Cookies          24576 bytes
```

`Cookies` é um banco **SQLite de verdade** — no CineFav vem vazio (login é mock, sem WebView autenticado), mas é **o mesmo arquivo** que guarda sessão real em qualquer app híbrido com login via WebView. A mecânica de extração é idêntica — só muda o que tem dentro.

O impacto, se tivesse conteúdo: esse cookie/token de sessão (ou JWT) é literalmente a prova de "eu já fiz login". Quem o copia **autentica como o usuário sem saber a senha** — dá pra reproduzir requisições em nome dele em qualquer sistema onde aquela sessão ainda for válida. Não precisa quebrar senha nenhuma: só copiar o que já prova que ela foi digitada.

Security — por que isso ainda importa em 2026

Isso rodou porque nosso build é `debuggable` (todo `assembleDebug` é) — Android 12+ passou a excluir dados de apps **não-debuggable** por padrão. Então "hacker rouba dados de qualquer app hoje" não é a história — mas "qualquer build de desenvolvimento continua exposto" é, e acabamos de provar.

Por que ainda importa mesmo em produção: (1) devices Android <12 ainda existem no parque instalado real. (2) MASVS/OWASP trata como controle de **baseline** — auditoria não pergunta só "dá pra explorar hoje".

Como um atacante chegaria nisso na prática: precisa de **acesso físico ao device com Depuração USB ligada** — sem senha, sem root, só o cabo. Cenário real: device de desenvolvimento perdido/roubado, ou esquecido numa assistência técnica com debugging já autorizado. Não é hipotético de aula — SDK terceiro real (EngageLab) expôs "millions of Android wallets" por misconfig parecido ([Microsoft Security Blog, abr/2026](#)).

Security — achado real no manifest (todo mundo, sem adb)

```
cd exercicios/03-maestro-e2e/pratica
cat android/app/src/main/AndroidManifest.xml \
  | grep -E "allowBackup|EXTERNAL_STORAGE|SYSTEM_ALERT"
```

Resultado real:

```
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<application ... android:allowBackup="true" ...>
```

Dois achados reais (motivo de cada um no próximo slide) — `allowBackup="true"` e 3 permissões (`READ/WRITE_EXTERNAL_STORAGE` , `SYSTEM_ALERT_WINDOW`) que o app não usa.

Sem device? Prova o achado no binário

Já baixaram o `CineFav.apk` pra instalar (COMECE-AQUI) — dá pra provar o mesmo achado **sem buildar nada**, direto no binário:

```
aapt dump xmltree CineFav.apk AndroidManifest.xml | grep -i backup
```

Resultado real:

```
A: android:allowBackup(0x01010280)=(type 0x12)0xffffffff
```

O manifest compilado não guarda texto "true"/"false", guarda um inteiro — `0xffffffff` = true (todos os bits ligados). Depois do fix (próximos slides), o mesmo comando mostra `0x00000000` (= false) — prova que pegou no binário.

Por que cada permissão não-usada é um risco

`SYSTEM_ALERT_WINDOW` — permissão histórica de **malware de overlay/tapjacking**: desenha uma tela por cima de outro app pra roubar toque ou senha (o app pensa que você tocou nele, mas tocou numa camada invisível). O CineFav não desenha overlay nenhum — não devia pedir isso.

`READ/WRITE_EXTERNAL_STORAGE` — não é só "ler os dados do próprio app": amplia a superfície pra **qualquer arquivo no storage compartilhado** do device (fotos, downloads de outros apps).

Least privilege: cada permissão concedida é uma porta a mais pra alguém explorar **se** o app for comprometido — mesmo que o CineFav em si seja confiável hoje.

allowBackup = OWASP M9 = OWASP M2 — mesmo achado, 2 rótulos

M9 é a numeração do **Mobile Top 10 2024** (atual). M2 é a numeração da lista **2016** (antiga). **Mesma categoria** (Insecure Data Storage) — 2 rótulos só porque o catálogo do OWASP foi atualizado entre as duas versões.

Isso não é só trivia: catálogos de vulnerabilidade mudam de numeração com o tempo (CWE, OWASP Top 10 web também já repaginou várias vezes). Saber **ler** um achado — identificar a categoria, não decorar o código — é o que generaliza pra qualquer lista nova que sair. Ref.: [OWASP MASTG](#).

Security — fix

```
- android:allowBackup="true"  
+ android:allowBackup="false"
```

- remover as 3 permissões não usadas do topo do manifest.

`allowBackup="false"` não é só "boa prática" — é o **Android recusando**, no nível do sistema operacional, qualquer tentativa de backup pro app. Enforcement real, não convenção.

Achar → corrigir → reverter. Quem tem device pode rebuildar (`./gradlew assembleDebug`) e reconferir com o mesmo `aapt dump` — passo completo no `lab-perf-security.md`. É literalmente o que MASVS/OWASP Mobile Top 10 pedem — vocês acabaram de fazer isso de verdade, sozinhos.

Recap — pirâmide de testes no CineFav

Camada	O que testamos	Como	Status
Manual/Exploratório	casos de teste funcionais, roteiro exploratório	execução manual, sem código	✅ (Atividade 1)
Unit	funções puras, stores Zustand	Jest, sem simulador	✅
Integração	fluxo entre componentes (RNTL, DOM emulado)	Jest + RNTL, mock	✅
E2E	app de verdade, abrindo no device	Maestro	agora

| Agora é a única camada que abre o app **real** — sem emular nada.

O que é Maestro, e por que a disciplina usa ele

Framework de teste **E2E mobile** — YAML declarativo, roda no **device/emulador real** (não emula DOM como o RNTL). Você descreve o comportamento esperado; ele interage com o app de verdade e confere o resultado.

Por que Maestro, e não Appium/Espresso/Detox (saíam do escopo desta disciplina):

- Sem código de automação (Java/Kotlin/JS) — YAML puro, mais rápido de escrever e ler
- Espera elemento aparecer automaticamente, sem `sleep()` manual (slide mais à frente)
- Mesma ferramenta cross-platform (Android/iOS) — não duplica suite

Espresso/Detox/XCUITest ainda existem no mercado — citados aqui só pra contexto (aparecem no Trabalho Final, tema Mobile Testing).

Maestro — anatomia de um flow

```
appId: com.puciec.cinefav      # qual app o flow abre
---
- launchApp                    # abre o app do zero
- tap0n:
    id: "login-email-input"    # toca no elemento com esse id
- inputText: "aluno@puc.br"    # digita no campo focado
- assertVisible:
    id: "movielist-screen"     # confirma que chegou na tela certa
```

Vocabulário mínimo pra ler (e escrever) qualquer flow: `appId` + `---` (header/separador), `launchApp` (abre o app), `tap0n` / `inputText` (interação), `assertVisible` (a asserção).

Isso é o `01-launch.yaml` de verdade, simplificado. Com esses 4 comandos vocês já leem qualquer um dos 5 flows da Atividade 4 — o resto (condicional, env, fragments) é variação em cima desse esqueleto, não coisa nova.

testIDs — a ponte entre RNTL e Maestro

No componente RN:

```
<Pressable testID={`movie-card-${movie.id}`} onPress={...}>
```

Vocês já usaram esse **mesmo testID** na Atividade 2/3 (RNTL):

```
screen.getByTestId('movie-card-603')
```

No Maestro:

```
- tapOn:  
  id: "movie-card-603"
```

Mesmo testID, três camadas. Isso não é coincidência — é a instrumentação de acessibilidade/teste do componente RN sendo reaproveitada.

tap0n: id: vs tap0n: "texto"

```
# Frágil – quebra se o texto mudar (i18n, dado dinâmico)
- tap0n: "Matrix"

# Estável – aponta pro elemento, não pro conteúdo
- tap0n:
  id: "movie-card-603"
```

Mesmo argumento de `getByTestId` vs `getByText` no RNTL. Prefira ID quando disponível.

Accessibility role — o mesmo elemento, duas lentes

```
- tapOn:  
  id: "movie-card-heart-603"
```

Testa que o elemento existe. Mas o **leitor de tela** de um usuário cego não vê ID — vê:

```
accessibilityRole="button"  
accessibilityLabel="Adicionar favorito"
```

Se o componente RN não tiver esses props, o teste de ID passa mas a **acessibilidade real falha**. Testar por ID garante que existe; testar por role/label garante que é **usável**.

Os 5 flows da Atividade 4

Flow	Testa	Conceito novo
01-launch.yaml	login + lista aparece	modelo resolvido 
02-search.yaml	buscar filme	env (SEARCH_TERM)
03-favorite.yaml	favoritar → conferir em Favoritos	assert cross-tela
04-detail.yaml	abrir detalhe, favoritar, voltar	navegação
05-js-dynamic.yaml	termo de busca via JS	evalScript

01 – 03 fazemos **juntos, ao vivo**. 04 – 05 + bônus fragment: solo, em casa (prazo 09/07).

Maestro Studio — o que é e por que facilita

Editor visual embutido no Maestro — abre ao lado do emulador/device e mostra a **árvore de elementos da tela em tempo real**, com o `id` de cada um disponível pra copiar.

Por que usar em vez de ler o código-fonte:

- Acha o `id` certo em segundos, **apontando e clicando** — não grepando `.tsx`
- Funciona mesmo sem acesso ao código (útil pra inspecionar app de terceiros)
- Reduz o erro mais comum: `tap0n: "texto"` frágil, por não achar o testID no source

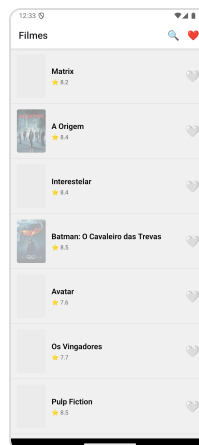
Reduz o atrito de escrever um flow novo ao mínimo: apontar → copiar o seletor → colar no YAML. O próximo slide mostra o "como" na prática.

Construindo 01-launch — o modelo

Já resolvido — mas construído **ao vivo** primeiro, pra ver o app abrir de verdade:

```
cd exercicios/03-maestro-e2e/pratica  
maestro test flows/01-launch.yaml
```

Celular físico? Pule `maestro-local.sh` (ele tenta bootar emulador) — vá direto no `maestro test`, com o cabo conectado.



Login mock (`aluno@puc.br` / `1234`) → tela de login some, lista de filmes aparece. Isso é o `assertVisible: id: "movielist-screen"` do flow passando.

01-launch — por que tem um `runFlow:` `when:` no meio

```
- launchApp
- runFlow:
  when:
    visible:
      id: "login-screen"
      file: _fragments/login.yaml
```

Por quê: se o app já estiver logado (rodou o flow antes, sessão não foi limpa), a tela de login não aparece — sem o `when:`, o flow tentaria tocar num elemento que não existe e quebraria. Com `when:`, só faz login **se precisar**. Testem: rodem `01-launch.yaml` duas vezes seguidas — na 2ª, o login é pulado sozinho.

Mesmo `id` do teste RNTL — `login-screen` é o mesmo `testID` que vocês já usam desde a Atividade 2. Não é coincidência.

Construindo 02-search — juntos

Scaffold que vocês recebem (`pratica/flows/02-search.yaml`) só tem os `TODO`:

```
appId: com.puciec.cinefav
env:
  SEARCH_TERM: "Matrix"
---
- launchApp
# TODO: faça login (copie os passos do flow 01)
# TODO: abra a busca — id "movielist-search-button"
# TODO: toque em "search-input" e use inputText com ${SEARCH_TERM}
# TODO: assertVisible do ${SEARCH_TERM}
```

Onde acham os `id` s? Não é adivinhação — abrimos o **Maestro Studio** agora.

Achando o seletor com Maestro Studio

```
maestro studio      # editor visual embutido no CLI – localhost:9999
```

Com o app aberto na tela de busca: **botão direito** no campo → menu mostra os comandos Maestro pra aquele elemento, com `id: "search-input"`. Botão direito no botão de busca do header → `id: "movielist-search-button"`. **Aponta e clica, não adivinha no código-fonte.**

Não é copiar-colar cego. O que sai do menu geralmente precisa de limpeza: prefere `tap0n: { id: ... }` a `tap0n: "texto"` (mais estável), remove passo redundante.

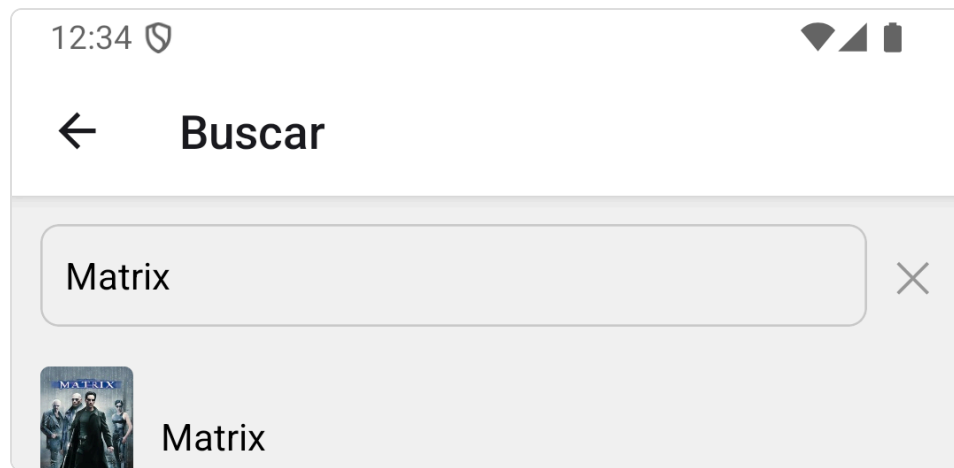
 [Run tests with Maestro Studio](#) — guia oficial do inspector.

02-search — completo e rodando

```
- launchApp
- runFlow:
  when:
    visible: { id: "login-screen" }
    file: _fragments/login.yaml
- tap0n: { id: "movielist-search-button" }
- tap0n: { id: "search-input" }
- inputText: ${SEARCH_TERM}
- assertVisible: ${SEARCH_TERM}
```

env: SEARCH_TERM: "Matrix" é o default.

02-search — rodando



Sobrescreve o termo na hora, sem duplicar arquivo:

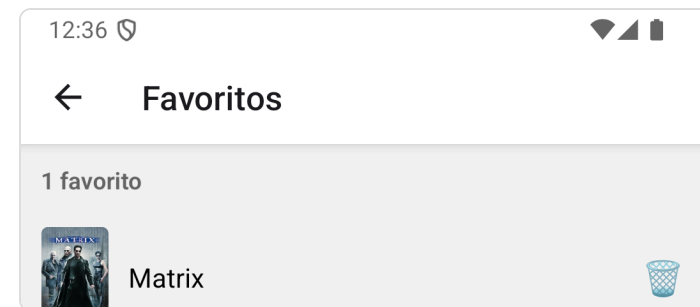
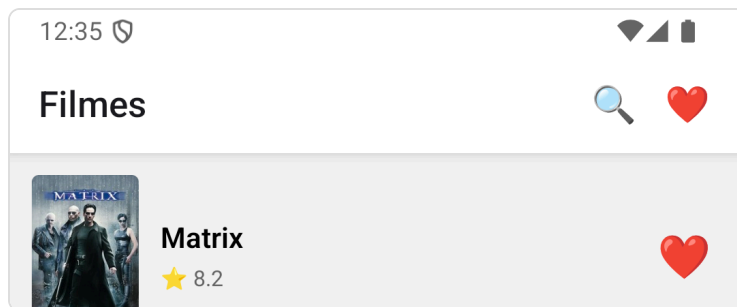
```
maestro test flows/02-search.yaml -e SEARCH_TERM=Avatar
```

Mesma ideia de parametrizar teste que vocês já usam em Jest (`it.each`) — um flow, dado diferente a cada execução.

Construindo 03-favorite — juntos

Conceito novo: **assert cross-tela** — ação numa tela (favoritar na lista) reflete em outra (Favoritos) + contador.

```
- launchApp
- runFlow:
    when: { visible: { id: "login-screen" } }
    file: _fragments/login.yaml
- tapOn: { id: "movie-card-heart-603" }           # favorita Matrix na lista
- tapOn: { id: "movielist-favorites-button" }     # abre Favoritos
- assertVisible: { id: "favorites-item-603" }
- assertVisible: "1 favorito"
```



Reparou que repetimos o login 3x? — Fragments

01, 02 e 03 chamaram o mesmo `runFlow: file: _fragments/login.yaml` — não é acidente:

```
# _fragments/login.yaml – 1 arquivo, chamado pelos 5 flows
appId: com.puciec.cinefav
---
- tap0n: { id: "login-email-input" }
- inputText: "aluno@puc.br"
- tap0n: { id: "login-password-input" }
- inputText: "1234"
- tap0n: { id: "login-submit-button" }
```

Se o fluxo de login mudar (novo campo, botão reposicionado), muda em **1 lugar**, não em 5 arquivos. Mesma ideia de "não repita conhecimento" que vocês já usam com função auxiliar no código.

Gancho pro Trabalho Final: **tema 2 (Arquitetura de Suíte)** leva esse reuso ainda mais longe — Robot Pattern organiza isso de forma mais estruturada.

Prévia do que espera vocês em 05-js-dynamic (casa)

04-detail (navegação) e 05-js-dynamic são solo, em casa. Rápido preview do 05 — JS embutido com evalScript:

```
- evalScript: ${output.termo = ['Matrix', 'Avatar', 'Interestelar'][Math.floor(Math.random()*3)]}  
- tap0n: { id: "movielist-search-button" }  
- tap0n: { id: "search-input" }  
- inputText: ${output.termo}  
- assertVisible: ${output.termo}
```

Escolhe um título **aleatório** e busca por ele — mesmo flow testa termos diferentes a cada execução. `output` é onde `evalScript` grava; `${output.termo}` lê esse valor depois, igual `${SEARCH_TERM}` do `env:` — mesma interpolação, JS só entra quando a lógica é complexa demais pra um valor fixo.

Por que Maestro não precisa de `sleep()` manual

Diferente de Selenium/Appium clássico, `tapOn / isVisible` do Maestro **já esperam** o elemento aparecer (timeout configurável) — não precisa de `sleep(2000)` espalhado pelo flow.

É por isso que E2E ainda assim pode ser flaky (próximo slide) — o wait automático ajuda com timing, mas não resolve tudo (rede lenta, animação, estado inesperado do device).

E2E é caro e às vezes flaky

Quando falha, o Maestro mostra exatamente onde:

```
x Unable to find element with text: "Spider-Man"
```

Rodar de novo — às vezes passa (flake real, aconteceu na última aula). **Retry existe por isso.** Diferente de unit/integração (determinístico), E2E depende do device/emulador de verdade.

Recap — o que construímos hoje x o que é com vocês

Flow	Hoje (ao vivo)	Recurso Maestro
01-launch.yaml	✅ construído junto	runFlow: when: (condicional) + fragment
02-search.yaml	✅ construído junto	env: + Maestro Studio
03-favorite.yaml	✅ construído junto	assert cross-tela
04-detail.yaml	🏠 casa (09/07)	navegação
05-js-dynamic.yaml	🏠 casa (09/07) — vimos preview	evalScript + \${output.campo}

Vocês não estavam só "copiando comando" — cada flow carrega um conceito, e vocês construíram os 3 primeiros com as próprias mãos. 04 e 05 repetem o mesmo padrão — é replicar o que já fizeram aqui.

Trabalho Final

60 pts · grupo de 3–4 · 1 dos 13 temas (5 trilhas)

13 temas, 5 trilhas — escolha pela skill, não só pela IA

Revisado hoje: temas contra evidência real de mercado. **Tema 2 mudou** (Espresso/XCUITest nunca foi praticado em aula) e **2 temas novos entraram** — cobrindo perfis além de "automation engineer puro".



Trilha Automação & Arquitetura

#	Tema	Stack
1	Automação Mobile E2E ★★★★★	Maestro + Cloud Devices (diferencial)
2	Arquitetura de Suíte de Teste ★★	Robot Pattern sobre RNTL e/ou runFlow do Maestro em camadas
5	Visual Regression Testing Mobile ★★	Maestro Visual Testing (nativo) ou React Native Owl
9	Mobile API Contract Testing ★★★★★	Pact + Jest (consumer-driven contracts)
13	POM (Page Object Model) em Maestro ★★★★★	JS output + runScript , seletor separado do flow

 Trilha IA aplicada a testes

#	Tema	Stack
6	Test Generation com LLM ★★ ★	Claude API + Maestro (ou Claude Code via MCP)
7	AI Agent Exploratory ★★ ★★	Claude Code + Maestro MCP (agentic exploration)

 Trilha Performance & Segurança

#	Tema	Stack
3	Performance Mobile Testing ★★ ★ — aprofunda o lab de hoje	Flashlight + Reassure (Callstack)
4	Mobile Security Testing ★★ ★ — aprofunda o lab de hoje	OWASP MASVS + achado manual (obrigatório) + MobSF (bônus)

 Trilha Qualidade & Pipeline

#	Tema	Stack
8	CI/CD Pipeline Mobile ★★★★★	GitHub Actions + Fastlane (build+test lanes)
11	Mutation Testing ★★★	Stryker (StrykerJS) sobre lib de domínio

Trilha Manual, Exploratório & Acessibilidade

#	Tema	Stack
10	Accessibility Testing Mobile ★★	Android Accessibility Scanner + auditoria manual WCAG
12	Manual → Regressão Automatizada ★★ ★	Parte A: heurística (TQED/Gamified) · Parte B: conversão RNTL/Maestro

Tema 12 é pra quem curte testar manualmente e não quer o projeto inteiro em código —
Parte A (manual) + Parte B (automação, avaliável em CI).

Onde ler tudo

Enunciado completo (com a coluna "por que é real" — fontes/evidência por tema):
`exercicios/04-trabalho-final/enunciado.md` (público, no repo).

Rubrica (60 pts)

Critério	Pts
Apresentação oral (10min + 5min Q&A)	12
Artefato técnico (repo funcional)	21
Relatório analítico (1pg)	12
Domínio conceitual em Q&A	9
Originalidade e profundidade	6

Entrega: 15/07/2026 (já no Canvas). **Data da apresentação: combinamos agora.**

Combinando a data da apresentação

Aula final = apresentações + IA em testes/CI-CD (conteúdo remanescente).

3 opções, todas depois da entrega (15/07) e da Final da Copa (19/07, sem conflito de jogo) — sem feriado nacional/estadual em MG nesse período:

Data	Dia
20/07	Segunda
21/07	Terça
23/07	Quinta

Falta só isso.